REST API

For Series 4 projectors

Reference Guide

ENABLING BRIGHT OUTCOMES

BARCO

Table of contents

1 REST API Introduction 7
1.1 Introduction 8
1.2 Swagger UI 9

2 Illumination REST API 11
2.1 Check actual power in percentage 12
2.2 CLO control 12
2.3 Power limitation check 12
2.4 Get maximum power value lasers in procentage 13
2.5 Get minimum power value lasers in percentage 13
2.6 Get power value in percentage 13
2.7 Set Power value 14
2.8 Power limit reason 14

3 Testpattern REST API 15
3.1 Testpattern ON/OFF status 17
3.2 Testpattern ON/OFF 17
3.3 Get selected testpattern name 17
3.4 Select testpattern 18
3.5 Iterate testpatterns 18
3.6 Iterate testpatterns in viewing order 18
3.7 Iterate testpatterns in viewing order 19
3.8 Testpattern available 19
3.9 Testpattern properties 20
3.10 Testpattern list 20
3.11 List of internal patterns 21
3.12 Testpattern set properties 21
3.13 Testpattern toggle 22
3.14 Size testpattern list 22
3.15 List possible testpatterns 23

4 Macro REST API 25
4.1 Execute macro 27
4.2 Execute macro's preset 27
4.3 Fetch last executed macro details 27
4.4 Fetch macro list 28

4.5 Fetch macro preset list ... 28
4.6 Create macro ... 28
4.7 Clear macro preset ... 29
4.8 Clear macro preset ... 29
4.9 Execute preset ... 29
4.10 Link macro to preset ... 30

5 Mediaplayer REST API ... 31
5.1 Mediaplayer availabilty status ... 32
5.2 Mediaplayer ready status ... 32
5.3 Mediaplayer serial number ... 32
5.4 Mediaplayer type ... 33
5.5 Mediaplayer reset ... 33

6 Network REST API ... 35
6.1 Domain name network ... 37
6.2 Hostname in network ... 37
6.3 Networking service version ... 37
6.4 Configuration method of device ... 38
6.5 General type of the network device ... 38
6.6 MAC address device ... 38
6.7 IP4 current configuration ... 39
6.8 Manual IP4 configuration ... 39
6.9 IP6 current configuration ... 40
6.10 Manual IP6 configuration ... 40
6.11 Current state device in network ... 41
6.12 Additional device state info ... 41

7 Lens REST API ... 43
7.1 Current selected lens ... 44
7.2 Current lens metadata ... 44
7.3 Set current lens metadata ... 44
7.4 Lens avalability ... 45
7.5 List of possible lenses ... 45
7.6 Select a lens ... 47

8 Dowser (shutter) REST API ... 49
8.1 Get dowser position ... 50
8.2 Get dowser state ... 50
8.3 Put dowser in a position ... 50
8.4 Get object path ... 51
8.5 Get state of dowser ... 51
8.6 Toggle dowser ... 51

9 System REST API ... 53
9.1 Projector article number ... 55
9.2 Colorwheel article number ... 55
9.3 Colorwheel name ... 55
9.4 Projector family name ... 56
9.5 Firmware version projector ... 56
9.6 Get initial state after startup ... 56
9.7 Set initial state after startup ... 57
9.8 Get model name projector ... 57
9.9 Get custom name projector ... 57

9.10 Set custom name projector 58
9.11 Get serial number projector 58
9.12 Get current state projector 59
9.13 Get the transition state of the device 59
9.14 Show activity 59
9.15 Get identification of projector 60
9.16 Get identifications of projector 60
9.17 Go into eco mode 61
9.18 Go to ready mode 61
9.19 Power on light 61
9.20 Power off light 62
9.21 Reboot complete projector 62

Index 63

REST API Introduction

1

1.1 Introduction

Overview

The Series 4 projectors support a Representation State Transfer Application Programming Interface (REST API) to control a number of parameters of the projector. ICMP(-X) is not in scope.

Examples

Different tools can be used to interact with the REST API. The examples in this manual will use the open source tool cURL (*http://curl.haxx.se/download.html*).

For example, to get the power state of projector, the following command can be used:

```
curl --user username:pwd http://<unit-ip>/rest/system/state
```

To show for example a test pattern, the following command can be used:

```
curl --user username:pwd http://<unit-ip>/rest/image/testpattern/show
```

Authentication

The majority of the API uses username and password as authentication.

Using curl, credentials can be passed along with the request. Each request requires credentials.

Get property value

Example: get the value of the property `system.state`

Using `curl` to get a property value

```
curl --user username:pwd http://<Unit-IP>/rest/system/state
```

The result is returned as a JSON string

```
{ "result":"ready" }
```

Note: when using curl, **GET** requests are default and need not be specified on the command line.

Set property value

Example: to set testpattern on/off `image.testpattern.show?value=true`

```
curl -X PUT --user username:pwd http://<unit-ip>/rest/image/testpattern/show?value=
true
```

Value return on success

```
{ "result": true}
```

Method call

Using `curl` to call method

to select a testpattern

```
curl -X PUT --user username:pwd http://<unit-IP>/rest/image/testpattern/selected -G
--data-urlencode "value=icp:Alternating Checkerboard"
```

The above example specifies the parameters to the function as a **JSON** string. Using -G and –dataurlencode makes curl encode the query string and append it to the URL. The result is returned as JSON.

```
{"result": true}
```

Parameter types

Type	Possible value
boolean	true or false
integer	positive or negative value

Type	Possible value
json	JSON format
string	any characters

Return Codes and Messages

When a REST command fails, an error code and message will be returned. Next to the generic error messages in the table below.

Code	Description
200	The REST API call was successful.
400	Error, the client request was malformed and could not be processed.
401	Authentication using the supplied credentials failed
404	The resource (property or method) was not found. Can be the result of misspelling a name or that authentication is required
503	The request could not be completed because of backend service availability

1.2 Swagger UI

About Swagger

Swagger is test interface where each REST API command can be seen and tested before it can be used in your own software.

Access Swagger

In the address line of you browser enter the following url: http://<unit-ip>/api-docs/ or https://<unit-ip>/api-docs/

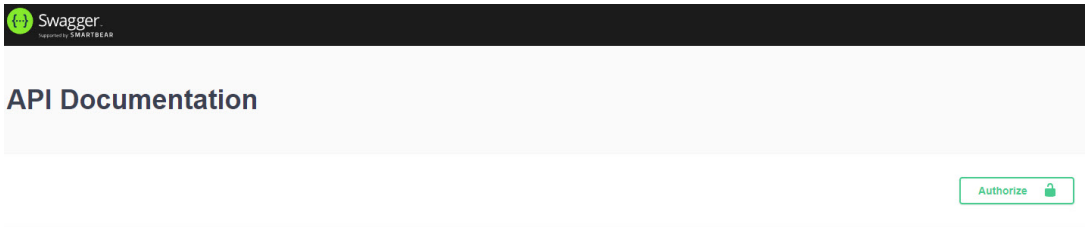

Image 1–1

You need to first authorize by just clicking on the **Authorize** button.

You will need to enter your credentials.

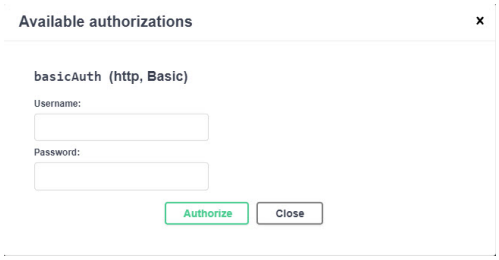

Image 1–2

By default:

- User: admin
- Password: Admin1234

How to use it

Search for the command you want to test and click on the downwards arrow.

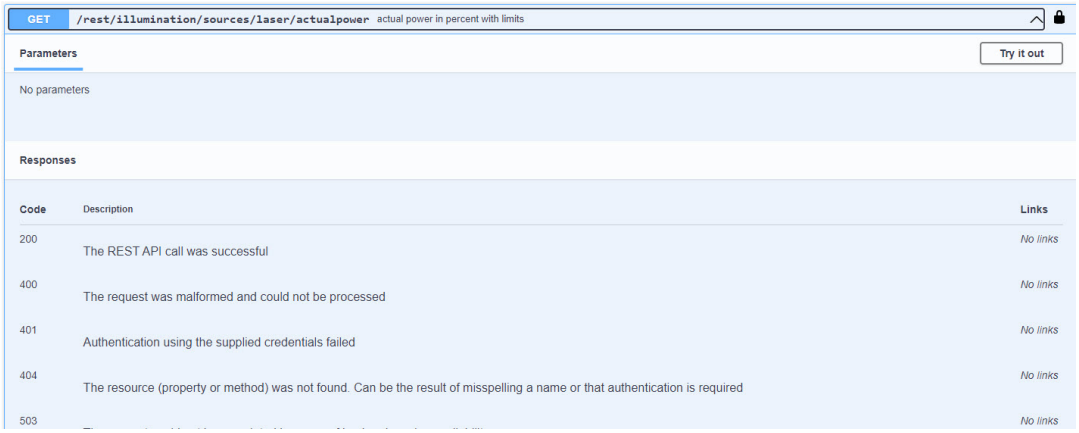

Image 1–3

Click on **Try it out**.

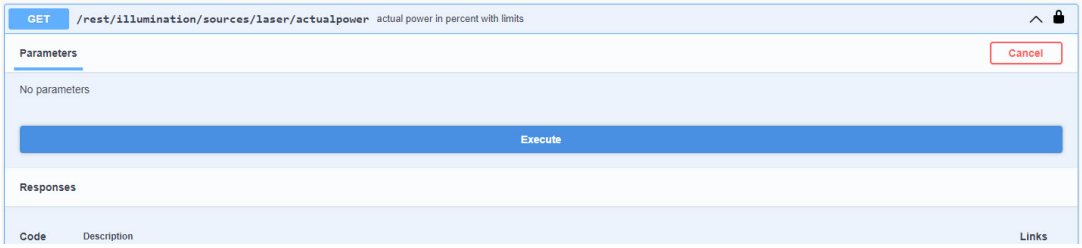

Image 1–4

Fill out the paramaters or values when necessary and click on **Execute**.

A result will be reported.

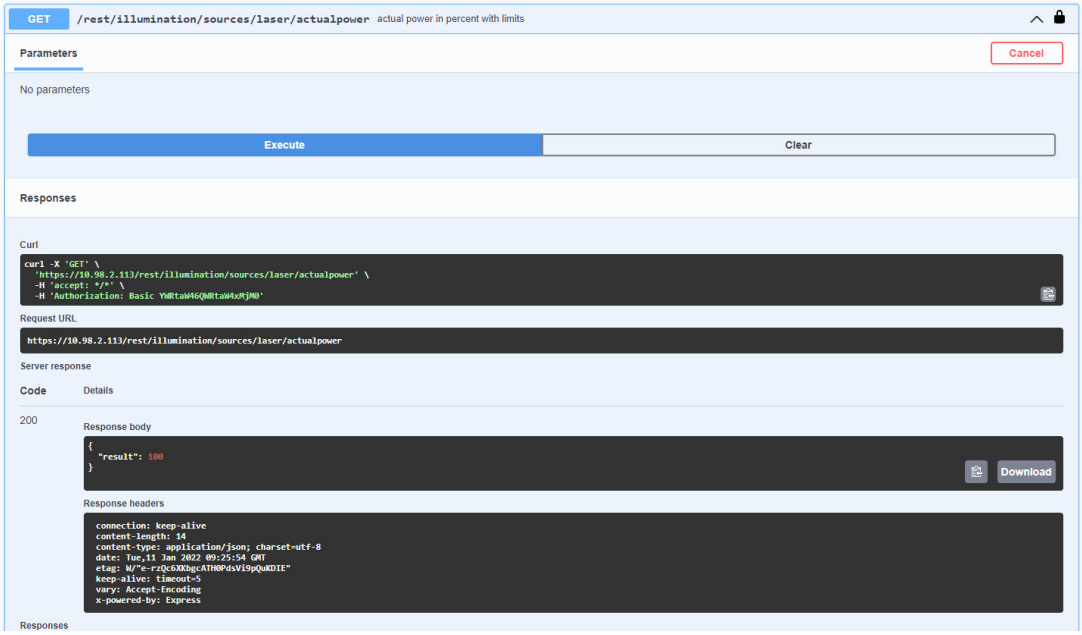

Image 1–5

Illumination REST API

2

2.1 Check actual power in percentage 12
2.2 CLO control 12
2.3 Power limitation check 12
2.4 Get maximum power value lasers in procentage 13
2.5 Get minimum power value lasers in percentage 13
2.6 Get power value in percentage 13
2.7 Set Power value 14
2.8 Power limit reason 14

Command overview

REST Command	Type	Parameter	Result (example)
/rest/illumination/sources/laser/actualpower	GET	-	{ "result":100 }
/rest/illumination/sources/laser/controlledbyclo	GET	-	{ "result":false }
/rest/illumination/sources/laser/ispowerlimited	GET	-	{ "result":false }
/rest/illumination/sources/laser/maxpower	GET	-	{ "result":100 }
/rest/illumination/sources/laser/minpower	GET	-	{ "result":55 }
/rest/illumination/sources/laser/power	GET	-	{ "result":100 }
	PUT	percentage	{ "result" : true }
/rest/illumination/sources/laser/powerlimitreason	GET	-	{ "result" : ”<reason>”}

2.1 Check actual power in percentage

API

/rest/illumination/sources/laser/actualpower

Type

GET

Parameter

-

Return value

Example:

```
{ "result":100 }
```

Example (with curl)

```
curl --user username: pwd http://<unit-ID>/rest/illumination/sources/laser/actualpower
```

2.2 CLO control

API

/rest/illumination/sources/laser/controlledbyclo

Type

GET

Parameter

-

Return value

Example:

`{ "result":false }` or `{ "result":true }`

Example (with curl)

```
curl --user username: pwd http://<unit-ID>/rest/illumination/sources/laser/controlledbyclo
```

2.3 Power limitation check

API

/rest/illumination/sources/laser/ispowerlimited

Type

GET

Parameter

-

Return value

Example:

`{ "result":false }` or `{ "result":true }`

Example (with curl)

```
curl --user username: pwd http://<unit-ID>/rest/illumination/sources/laser/ispowerlimited
```

2.4 Get maximum power value lasers in procentage

API

/rest/illumination/sources/laser/maxpower

Type

GET

Parameter

-

Return value

Example:

`{ "result":92 }`

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/illumination/sources/laser/maxpower
```

2.5 Get minimum power value lasers in percentage

API

/rest/illumination/sources/laser/minpower

Type

GET

Parameter

-

Return value

Example:

`{ "result":55 }`

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/illumination/sources/laser/minpower
```

2.6 Get power value in percentage

API

/rest/illumination/sources/laser/power

Type

GET

Parameter

-

Return value

```
{ "result" : 100 }
```

Example (with curl)

```
curl --user username:pwd http:///rest/illumination/sources/laser/power
```

2.7 Set Power value

API

/rest/illumination/sources/laser/power

Type

PUT

Parameter

Example: `value=22`

Return value

```
{ "result" : true }
```

Example (with curl)

```
curl --user username: pwd -X PUT http:///rest/illumination/sources/laser/power?
value=22
```

2.8 Power limit reason

API

/rest/illumination/sources/laser/powerlimitreason

Type

GET

Parameter

-

Return value

```
{ "result" : ”<reason>”}
```

Example (with curl)

```
curl --user username:pwd http:///rest/illumination/sources/laser/powerlimitreason
```

Testpattern REST API

3

3.1 Testpattern ON/OFF status 17
3.2 Testpattern ON/OFF 17
3.3 Get selected testpattern name 17
3.4 Select testpattern 18
3.5 Iterate testpatterns 18
3.6 Iterate testpatterns in viewing order 18
3.7 Iterate testpatterns in viewing order 19
3.8 Testpattern available 19
3.9 Testpattern properties 20
3.10 Testpattern list 20
3.11 List of internal patterns 21
3.12 Testpattern set properties 21
3.13 Testpattern toggle 22
3.14 Size testpattern list 22
3.15 List possible testpatterns 23

Command overview

REST Command	Type	Parameter	Result (example)
/rest/image/testpattern/show	GET	-	{ "result": true}
/rest/image/testpattern/show	PUT	value=true or false	{ "result": true}
/rest/image/testpattern/selected	GET	-	{ "result": "icp: Alternating Checkerboard" }
/rest/image/testpattern/selected	PUT	value=<tp-name>	{ "result": true}
/rest/image/testpattern/iterator/button/next	POST	-	{"result": {}}
/rest/image/testpattern/iterator/button/list	GET	value=<tp-name>	{"result": {}}
	PUT	-	{ "result": "<list>" }
/rest/image/testpattern/available	GET	-	{ "result": true}

REST Command	Type	Parameter	Result (example)
/rest/image/testpattern/getproperties	POST	params={ "id": “<testpattern-name>” }	{"result": {}}
/rest/image/testpattern/list	POST	-	{ "result": "<list>" }
/rest/image/testpattern/listinternal	POST	-	{ "result": "<list>" }
/rest/image/testpattern/setproperties	POST	params={ "id": “<testpattern-name>”, “properties”: “<property-type” }	{"result": {}}
/rest/image/testpattern/toggle	POST	-	{"result": {}}
/rest/image/testpattern/iterator/button/size	GET	-	{ "result": 9}
/rest/image/testpattern/iterator/button/listpossiblepatterns	POST	-	{ "result": "<list>" }

3.1 Testpattern ON/OFF status

API

/rest/image/testpattern/show

Type

GET

Parameter

-

Return value

```
{ "result": true}
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/image/testpattern/show
```

3.2 Testpattern ON/OFF

API

/rest/image/testpattern/show

Type

PUT

Parameter

```
value=true
```

Return value

```
{ "result": true}
```

Example (with curl)

```
curl -X PUT --user username:pwd http://<unit-ID>/rest/image/testpattern/show?value=
true
```

3.3 Get selected testpattern name

API

/rest/image/testpattern/selected

Type

GET

Parameter

-

Return value

Example: `{ "result": "icp:Alternating Checkerboard" }`

Example (with curl)

```
curl -X --user username:pwd http://<unit-ID>/rest/image/testpattern/selected
```

3.4 Select testpattern

API

/rest/image/testpattern/selected

Type

PUT

Parameter

`value=<tp-name>`

Example : `value=icp:Alternating Checkerboard`

Return value

```
{"result": true}
```

Example (with curl)

```
curl -X PUT --user username:pwd http://<unit-ID>/rest/image/testpattern/selected -G
--data-urlencode "value=icp:Alternating Checkerboard"
```

3.5 Iterate testpatterns

API

/rest/image/testpattern/iterator/button/next

Type

POST

Parameter

-

Return value

```
{"result": {}}
```

Example (with curl)

```
curl -X POST --user usrname: pwd http://<unit-ID>/rest/image/testpattern/iterator/
button/next
```

3.6 Iterate testpatterns in viewing order

API

/rest/image/testpattern/iterator/button/list

Type

GET

Parameter

Example:

```
value=FocusGreen
```

Return value

```
{}
```

Example (with curl)

```
curl -X POST --user usrname:pwd http://<unit-ID>/rest/image/testpattern/iterator/
button/list?value=FocusGreen
```

3.7 Iterate testpatterns in viewing order

API

/rest/image/testpattern/iterator/button/list

Type

PUT

Parameter

-

Return value

```
{   "result": [
                        "Framing_uncorrected",
                        "FocusGreen",
                        "RGB-12bit-Full Screen White",
                        "RGB-12bit-Color Bars"
        ]
}
```

Example (with curl)

```
curl -X POST --user usrname:pwd http://<unit-ID>/rest/image/testpattern/iterator/
button/list
```

3.8 Testpattern available

API

/rest/image/testpattern/available

Type

GET

Parameter

-

Return value

```
{"result":true}
```

Example (with curl)

```
curl -X GET --user usrname:pwd http://<unit-ID>/rest/image/testpattern/available
```

3.9 Testpattern properties

API

/rest/image/testpattern/getproperties

Type

POST

Parameter

String

params={ "id": “<testpattern-name>” }

Return value

```
{"result": {}}
```

Example (with curl)

```
curl -X POST --user usrname: pwd http://<unit-ID>/rest/image/testpattern/
getproperties --data-urlencode ’params={ "id": “<testpattern-name>” }’
```

3.10 Testpattern list

API

/rest/image/testpattern/list

Type

POST

Parameter

-

Return value

Example:

```
{  "result": [
    {
        "name": "DualStack3D_Left",
        "location": "standard:icp",
        "id": "icp:DualStack3D_Left",
        "above": false,
        "internal": false,
        "properties": {"accesslevel": "EndUser"}
   },
   {
        "name": "RGB-12bit-3D Dynamic Range",
        "location": "standard:icp",
        "id": "icp:RGB-12bit-3D Dynamic Range",
        "above": false,
        "internal": false,
        "properties": {"accesslevel": "EndUser"}
   }
```

```
}
]
```

Example (with curl)

```
curl -X POST --user usrname:pwd http://<unit-ID>/rest/image/testpattern/list
```

3.11 List of internal patterns

API

/rest/image/testpattern/listinternal

Type

POST

Parameter

-

Return value

Example:

```
{  "result": [
    {
        "name": "DualStack3D_Left",
        "location": "standard:icp",
        "id": "icp:DualStack3D_Left",
        "above": false,
        "internal": false,
        "properties": {"accesslevel": "EndUser"}
   },
   {
        "name": "RGB-12bit-3D Dynamic Range",
        "location": "standard:icp",
        "id": "icp:RGB-12bit-3D Dynamic Range",
        "above": false,
        "internal": false,
        "properties": {"accesslevel": "EndUser"}
   }
}
]
```

Example (with curl)

```
curl -X POST --user usrname: pwd http://<unit-ID>/rest/image/testpattern/
listinternal
```

3.12 Testpattern set properties

API

/rest/image/testpattern/setproperties

Type

POST

Parameter

Query

params={ "id": “<testpattern-name>”, “properties”: “<property-type” }

Return value

Example:

```
{"result": {}}
```

Example (with curl)

```
curl -X POST --user usrname: pwd http://<unit-ID>/rest/image/testpattern/setproperties --data-urlencode ’params= {"id":“<testpattern-name>”, “properties”: “<property-type” }’
```

3.13 Testpattern toggle

API

/rest/image/testpattern/toggle

Type

POST

Parameter

-

Return value

Example:

```
{"result": {}}
```

Example (with curl)

```
curl -X POST --user usrname:pwd http://<unit-ID>/rest/image/testpattern/toggle
```

3.14 Size testpattern list

API

/rest/image/testpattern/iterator/button/size

Type

GET

Parameter

-

Return value

Example:

```
{"result": 9 }
```

Example (with curl)

```
curl -X POST --user usrname: pwd http://<unit-ID>/rest/image/testpattern/iterator/button/size
```

3.15 List possible testpatterns

API

/rest/image/testpattern/iterator/button/listpossiblepatterns

Type

POST

Parameter

-

Return value

Example:

```
{ "result": [
            "DualStack3D_Left",
            "RGB-12bit-3D Dynamic Range",
            "FocusWhite",
            "RGB-12bit-Full Screen Magenta",
            "Focus",
            "Convergence_pattern_xxB",
            "Convergence_pattern_xxC",
            "Convergence_pattern_xxS",
            ....
        ]
}
```

Example (with curl)

```
curl -X POST --user usrname: pwd http://<unit-ID>/rest/image/testpattern/iterator/
button/listpossiblepatterns
```

Macro REST API 4

4.1 Execute macro 27
4.2 Execute macro's preset 27
4.3 Fetch last executed macro details 27
4.4 Fetch macro list 28
4.5 Fetch macro preset list 28
4.6 Create macro 28
4.7 Clear macro preset 29
4.8 Clear macro preset 29
4.9 Execute preset 29
4.10 Link macro to preset 30

Command overview

REST Command	Type	Parameter	Result (example)
/rest/macro/execute	POST	params={"name": "<macroname>"}	{ "result":{} }
/rest/macro/presets/execute	POST	params={"id": <preset-id>}	{ "result":{} }
/rest/macro/history	GET	-	{ "result": { "name": "test", ... } }
/rest/macro/list	GET	-	{ "result": ["Test1", "Test2", ...] }
/rest/macro/presets/list	GET	-	{ "result": [[1, "test"], ...] }
/rest/macro/create	POST	params={ "name": "<new-macroname>" }	{ "result": "<new-macroname>" }
/rest/macro/presets/clear	POST	params={ "id": <preset-id> }	{ "result": {} }
/rest/macro/presets/clearall	POST	-	{ "result": {} }

REST Command	Type	Parameter	Result (example)
/rest/macro/presets/execute	POST	params={ “id”: <preset-id> }	{ "result": {} }
/rest/macro/presets/link	POST	params={ “id”: <preset-id>, “name”: “<macro-name>”}	{ "result": {} }

4.1 Execute macro

API

/rest/macro/execute

Type

POST

Parameter

```
params={"name": "<macroname>"}
```

Return value

```
{ "result": { } }
```

Example (with curl)

```
curl --user username: pwd -X POST -G http://<unit-ID>/rest/macro/execute --data-urlencode 'params={"name": "<macroname>"}'
```

4.2 Execute macro's preset

API

/rest/macro/presets/execute

Type

POST

Parameter

```
params={"id": <preset-id>}
```

Return value

```
{ "result": { } }
```

Example (with curl)

```
curl --user username: pwd -X POST -G http://<unit-ID</rest/macro/presets/execute --data-urlencode 'params={"id": 2}'
```

4.3 Fetch last executed macro details

API

/rest/macro/history

Type

GET

Parameter

–

Return value

```
{ "result": { "name": "test", ... } }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/macro/history
```

4.4 Fetch macro list

API

/rest/macro/list

Type

GET

Parameter

-

Return value

```
{ "result": ["Test1", "Test2", ...] }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/macro/list
```

4.5 Fetch macro preset list

API

/rest/macro/presets/list

Type

GET

Parameter

-

Return value

```
{ "result": [[1, "test"], ...] }
```

Example (with curl)

```
curl --user username:pwd -X GET http://<unit-ID>/rest/macro/presets/list
```

4.6 Create macro

API

/rest/macro/create

Type

POST

Parameter

string

params={ “name”: “<new-macroname>” }

Return value

```
{ "result": "<new-macroname>" }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/macro/create --data-urlencode
'params={"name":"<new-macroname>"}'
```

4.7 Clear macro preset

API

/rest/macro/presets/clear

Type

POST

Parameter

query

params={ “id”: <preset-id> }

Return value

```
{ "result": {} }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/macro/presets/clear --data-
urlencode 'params={"id":<preset-id>}'
```

4.8 Clear macro preset

API

/rest/macro/presets/clearall

Type

POST

Parameter

-

Return value

```
{ "result": {} }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/macro/presets/clearall
```

4.9 Execute preset

API

/rest/macro/presets/execute

Type

POST

Parameter

query

params={ “id”: <preset-id> }

Return value

```
{ "result": {} }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/macro/presets/execute --data-urlencode 'params={“id”:<preset-id>}'
```

4.10 Link macro to preset

API

/rest/macro/presets/link

Type

POST

Parameter

query

params={ “id”: <preset-id>, “name”: “<macro-name>”}

Return value

```
{ "result": {} }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/macro/presets/execute --data-urlencode 'params={“id”:<preset-id>,“name”:“<macro-name>”}'
```

Mediaplayer REST API

5

5.1 Mediaplayer availabilty status 32
5.2 Mediaplayer ready status 32
5.3 Mediaplayer serial number 32
5.4 Mediaplayer type 33
5.5 Mediaplayer reset 33

Command overview

REST Command	Type	Parameter	Result (example)
/rest/mediaplayer/available	GET	-	{ "result": true }
/rest/mediaplayer/ready	GET	-	{ "result": true }
/rest/mediaplayer/serialnumber	GET	-	{ "result": “<serial-number>” }
/rest/mediaplayer/type	GET	-	{ "result": “Barco ICMP” }
/rest/mediaplayer/reset	POST	-	{ "result": {}}

5.1 Mediaplayer availabilty status

API

/rest/mediaplayer/available

Type

GET

Parameter

-

Return value

```
{ "result": true }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/mediaplayer/available
```

5.2 Mediaplayer ready status

API

/rest/mediaplayer/ready

Type

GET

Parameter

-

Return value

```
{ "result": true }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/mediaplayer/ready
```

5.3 Mediaplayer serial number

API

/rest/mediaplayer/serialnumber

Type

GET

Parameter

-

Return value

```
{ "result": "<serial-number>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/mediaplayer/serialnumber
```

5.4 Mediaplayer type

API

/rest/mediaplayer/type

Type

GET

Parameter

-

Return value

```
{ "result": "Barco ICMP" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/mediaplayer/type
```

5.5 Mediaplayer reset

API

/rest/mediaplayer/reset

Type

POST

Parameter

-

Return value

```
{ }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/mediaplayer/reset
```

Network REST API 6

6.1 Domain name network 37
6.2 Hostname in network 37
6.3 Networking service version 37
6.4 Configuration method of device 38
6.5 General type of the network device 38
6.6 MAC address device 38
6.7 IP4 current configuration 39
6.8 Manual IP4 configuration 39
6.9 IP6 current configuration 40
6.10 Manual IP6 configuration 40
6.11 Current state device in network 41
6.12 Additional device state info 41

Command overview

REST Command	Type	Parameter	Result (example)
/rest/network/domain	GET	-	{ "result": “Barco ICMP” }
/rest/network/hostname	GET	-	{ "result": “<hostname>” }
/rest/network/version	GET	-	{ "result": “<version>” }
/rest/network/device/lan/configuration	GET	-	{ "result": “MANUAL” }
/rest/network/device/lan/devicetype	GET	-	{ "result": “WIRED” }
/rest/network/device/lan/hwaddress	GET	-	{ "result": “<MAC-address>” }
/rest/network/device/lan/ip4config	GET	-	{ "result": “<IP-config>” }
/rest/network/device/lan/ip4configmanual	GET	-	{ "result": “<IP-config>” }

REST Command	Type	Parameter	Result (example)
/rest/network/device/lan/ip4config	GET	-	{ "result": “<IP6-config>” }
/rest/network/device/lan/ip6configmanual	GET	-	{ "result": “<IP6-config>” }
/rest/network/device/lan/state	GET	-	{ "result": “CONNECTED” }
/rest/network/device/lan/stateinfo	GET	-	{ "result": “” }

6.1 Domain name network

API

/rest/network/domain

Type

GET

Parameter

-

Return value

```
{ "result": "Barco ICMP" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/domain
```

6.2 Hostname in network

API

/rest/network/hostname

Type

GET

Parameter

-

Return value

```
{ "result": "<hostname>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/hostname
```

6.3 Networking service version

API

/rest/network/version

Type

GET

Parameter

-

Return value

```
{ "result": "<version>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/version
```

6.4 Configuration method of device

API

/rest/network/device/lan/configuration

Type

GET

Parameter

-

Return value

```
{ "result": "MANUAL" }
```

Result can be MANUAL or AUTO

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/configuration
```

6.5 General type of the network device

API

/rest/network/device/lan/devicetype

Type

GET

Parameter

-

Return value

```
{ "result": "WIRED" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/devicetype
```

6.6 MAC address device

API

/rest/network/device/lan/hwaddress

Type

GET

Parameter

-

Return value

```
{ "result": "<MAC-address>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/hwaddress
```

6.7 IP4 current configuration

API

/rest/network/device/lan/ip4config

Type

GET

Parameter

-

Return value

```
{
  "result": {
    "Address": "10.98.2.28",
    "Mask": "255.255.254.0",
    "Gateway": "10.98.2.1",
    "NameServers": "10.98.224.11"
  }
}
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/ip4config
```

6.8 Manual IP4 configuration

API

/rest/network/device/lan/ip4configmanual

Type

GET

Parameter

-

Return value

```
{
  "result": {
    "Address": "10.98.2.28",
    "Mask": "255.255.254.0",
    "Gateway": "10.98.2.1",
    "NameServers": "10.98.224.11"
  }
}
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/ip4configmanual
```

6.9 IP6 current configuration

API

/rest/network/device/lan/ip4config

Type

GET

Parameter

-

Return value

```
{
  "result": {
    "Address": " ",
    "Prefix": "0",
    "Gateway": "",
    "NameServers": ""
  }
}
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/ip6config
```

6.10 Manual IP6 configuration

API

/rest/network/device/lan/ip6configmanual

Type

GET

Parameter

-

Return value

```
{
  "result": {
    "Address": "::ffff:0:0 ",
    "Prefix": "96",
    "Gateway": "fe80::1",
    "NameServers": ""
  }
}
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/ip6configmanual
```

6.11 Current state device in network

API

/rest/network/device/lan/state

Type

GET

Parameter

-

Return value

```
{ "result": "CONNECTED" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/state
```

6.12 Additional device state info

API

/rest/network/device/lan/stateinfo

Type

GET

Parameter

-

Return value

Result can be empty.

```
{ "result": "" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/network/device/lan/stateinfo
```

Lens REST API 7

7.1 Current selected lens 44
7.2 Current lens metadata 44
7.3 Set current lens metadata 44
7.4 Lens avalability 45
7.5 List of possible lenses 45
7.6 Select a lens 47

Command overview

REST Command	Type	Parameter	Result (example)
/rest/optics/lens/lens	GET	-	{ "result": “<lens-id-description>” }
/rest/optics/lens/metadata	GET	-	{ "result": “<metadata>” }
	PUT	value=<value-property>	{ "result": { } }
/rest/optics/lens/selectavailable	GET	-	{ "result": true }
/rest/optics/lens/list	POST	-	{ "result": “<list>” }
/rest/optics/lens/select	POST	params={”lensid”: “<lens-article-number>”}	{ "result": true }

7.1 Current selected lens

API

/rest/optics/lens/lens

Type

GET

Parameter

-

Return value

```
{
  "result": {
    "Id": "<lens-article-number",
    "Description": "<lens-description-trow-ratio>"
  }
}
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/optics/lens/lens
```

7.2 Current lens metadata

API

/rest/optics/lens/metadata

Type

GET

Parameter

-

Return value

```
{"result": {
    <metadata>}}
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/optics/lens/metadata
```

7.3 Set current lens metadata

API

/rest/optics/lens/metadata

Type

PUT

Parameter

Key value string

value=<value-property>

Return value

```
{}
```

Example (with curl)

```
curl --user username:pwd -X PUT http://<unit-ID>/rest/optics/lens/metadata?value=
<value-property>
```

7.4 Lens avalability

API

/rest/optics/lens/selectavailable

Type

GET

Parameter

-

Return value

```
{ "result": true }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/optics/lens/selectavailable
```

7.5 List of possible lenses

API

/rest/optics/lens/list

Type

POST

Parameter

-

Return value

Example:

```
{
  "result": [
    {
      "Id": "Non-motorized lens",
      "Description": "Non-motorized lens"
    },
    {
      "Id": "R9801414",
      "Description": "TLD+ ULTRA LENS 0.85 -1.24"
    },
    {
      "Id": "R9855931",
      "Description": "MINOLTA(R) 0.98\" DC2K 1.4-2.05"
```

```
    },
    {
      "Id": "R98559311",
      "Description": "MINOLTA(R) 0.98\" DC2K 1.4-2.05 MAN"
    },
    {
      "Id": "R9855932",
      "Description": "FUJINON(R) 0.98\" DC2K 1.6-2.5"
    },
    {
      "Id": "R98559321",
      "Description": "FUJINON(R) 0.98\" DC2K 1.6-2.5 MAN"
    },
    {
      "Id": "R9855933",
      "Description": "MINOLTA(R) 0.98\" DC2K 1.6-2.5"
    },
    {
      "Id": "R98559331",
      "Description": "MINOLTA(R) 0.98\" DC2K 1.6-2.5 MAN"
    },
    {
      "Id": "R9855934",
      "Description": "FUJINON(R) 0.98\" DC2K 1.95-3.2"
    },
    {
      "Id": "R98559341",
      "Description": "FUJINON(R) 0.98\" DC2K 1.95-3.2 MAN"
    },
    {
      "Id": "R9855935",
      "Description": "KYOCERA(R) 0.98\" DC2K 1.95-3.2"
    },
    {
      "Id": "R98559351",
      "Description": "KYOCERA(R) 0.98\" DC2K 1.95-3.2 MAN"
    },
    {
      "Id": "R9855936",
      "Description": "MINOLTA(R) 0.98\" DC2K 2.4-3.9"
    },
    {
      "Id": "R98559361",
      "Description": "MINOLTA(R) 0.98\" DC2K 2.4-3.9 MAN"
    },
    {
      "Id": "R9855957",
      "Description": "FUJINON(R) 0.98\" DC2K 1.2-1.81"
    },
    {
      "Id": "R98559571",
      "Description": "FUJINON(R) 0.98\" DC2K 1.2-1.81 MAN"
    },
    {
      "Id": "R98559572",
      "Description": "MINOLTA(R) 0.98\" DC2K 1.2-1.81"
    },
    {
      "Id": "R9862005",
      "Description": "TLD+ ULTRA LENS 1.16-1.49"
    }
  ]
}
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/optics/lens/list
```

7.6 Select a lens

API

/rest/optics/lens/select

Type

POST

Parameter

params={”lensid”: “<lens-article-number>”}

Return value

```
{ "result": true }
```

Example (with curl)

```
curl -- user username: pwd -X POST http://<unit-ID>/rest/optics/lens/select --data-urlencode ’params={”lensid”: “<lens-article-number>”}’
```

Dowser (shutter) REST API

8

8.1 Get dowser position 50
8.2 Get dowser state 50
8.3 Put dowser in a position 50
8.4 Get object path 51
8.5 Get state of dowser 51
8.6 Toggle dowser 51

Command overview

REST Command	Type	Parameter	Result (example)
/rest/optics/shutter/position	GET	-	{ "result": "Open" }
/rest/optics/shutter/target	GET	-	{ "result": "Open" }
/rest/optics/shutter/position	PUT	value= <desired-position>	{ "result": "Open" }
/rest/optics/shutter/getobjectpath	POST	-	{ "result": "<object-path>" }
/rest/optics/shutter/getstate	POST	-	{ "result": "Stopped" }
/rest/optics/shutter/toggle	POST	-	{ "result": {} }

8.1 Get dowser position

API

/rest/optics/shutter/position

Type

GET

Parameter

-

Return value

`{ "result": "Open" }` or `{ "result": "Closed" }`

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/optics/shutter/position
```

8.2 Get dowser state

API

/rest/optics/shutter/target

Type

GET

Parameter

-

Return value

`{ "result": "Open" }` or `{ "result": "Closed" }`

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/optics/shutter/target
```

8.3 Put dowser in a position

API

/rest/optics/shutter/position

Type

PUT

Parameter

value= <desired-position>

Return value

`{ "result": "Open" }` or `{ "result": "Closed" }`

Example (with curl)

```
curl --user username: pwd -X PUT http://<unit-ID>/rest/optics/shutter/position?
value=<desired-position>
```

8.4 Get object path

API

/rest/optics/shutter/getobjectpath

Type

POST

Parameter

-

Return value

```
{ "result": "<object-path>" }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/optics/shutter/getobjectpath
```

8.5 Get state of dowser

API

/rest/optics/shutter/getstate

Type

POST

Parameter

-

Return value

Example

```
{ "result": "Stopped" }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/optics/shutter/getstate
```

8.6 Toggle dowser

API

/rest/optics/shutter/toggle

Type

POST

Parameter

-

Return value

Example

```
{ "result": {} }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/optics/shutter/toggle
```

System REST API 9

9.1 Projector article number 55
9.2 Colorwheel article number 55
9.3 Colorwheel name 55
9.4 Projector family name 56
9.5 Firmware version projector 56
9.6 Get initial state after startup 56
9.7 Set initial state after startup 57
9.8 Get model name projector 57
9.9 Get custom name projector 57
9.10 Set custom name projector 58
9.11 Get serial number projector 58
9.12 Get current state projector 59
9.13 Get the transition state of the device 59
9.14 Show activity 59
9.15 Get identification of projector 60
9.16 Get identifications of projector 60
9.17 Go into eco mode 61
9.18 Go to ready mode 61
9.19 Power on light 61
9.20 Power off light 62
9.21 Reboot complete projector 62

Command overview

REST Command	Type	Parameter	Result (example)
/rest/system/articlenumber	GET	-	{ "result": “<article-number>” }
/rest/system/colorwheel	GET	-	{ "result": “<colorwheel-article-number>” }
/rest/system/colorwheelname	GET	-	{ "result": “<family-name>” }
/rest/system/familyname	GET	-	{ "result": “<family-name>” }

REST Command	Type	Parameter	Result (example)
/rest/system/firmwareversion	GET	-	{ "result": “<firmware-version>” }
/rest/system/initialstate	GET	-	{ "result": “<initial-state>” }
	PUT	value= <initial-state>	{ "result": {}}
/rest/system/modelname	GET	-	{ "result": “<model-name>” }
/rest/system/name	GET	-	{ "result": “<custom-name>” }
	PUT	vaue=<custom-name>	{ "result": {}}
/rest/system/serialnumber	GET	-	{ "result": “<serial-number>” }
/rest/system/state	GET	-	{ "result": “<current-state>” }
/rest/system/targetstate	GET	-	{ "result": “<transition-state>” }
/rest/system/activity	POST	-	{ "result": “<activity>” }
/rest/system/getidentification	POST	params= {"identification": “<identification-string>”}	{ "result": “<identification-result>” }
/rest/system/getidentifications	POST	-	{ "result": “<list>” }
/rest/system/gotoeco	POST	-	{ "result": {}}
/rest/system/gotoready	POST	-	{ "result": {}}
/rest/system/poweron	POST	-	{ "result": {}}
/rest/system/poweroff	POST	-	{ "result": {}}
/rest/system/reboot	POST	-	{ "result": {}}

9.1 Projector article number

API

/rest/system/articlenumber

Type

GET

Parameter

-

Return value

```
{ "result": "<article-number>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/articlenumber
```

9.2 Colorwheel article number

API

/rest/system/colorwheel

Type

GET

Parameter

-

Return value

```
{ "result": "<colorwheel-article-number>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/colorwheel
```

9.3 Colorwheel name

API

/rest/system/colorwheelname

Type

GET

Parameter

-

Return value

```
{ "result": "<colorwheel-name>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/colorwheelname
```

9.4 Projector family name

API

/rest/system/familyname

Type

GET

Parameter

-

Return value

```
{ "result": "<family-name>" }
```

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/familyname
```

9.5 Firmware version projector

API

/rest/system/firmwareversion

Type

GET

Parameter

-

Return value

```
{ "result": "<firmware-version>" }
```

e.g. : 1.6.0–dev-n1795

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/firmwareversion
```

9.6 Get initial state after startup

API

/rest/system/initialstate

Type

GET

Parameter

-

Return value

```
{ "result": "<initial-state>" }
```

e.g. : ready

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/initialstate
```

9.7 Set initial state after startup

API

/rest/system/initialstate

Type

PUT

Parameter

value= <initial-state>

Return value

```
{}
```

Example (with curl)

```
curl --user username: pwd -X PUT http://<unit-ID>/rest/system/initialstate? value=
<intial-state>
```

9.8 Get model name projector

API

/rest/system/modelname

Type

GET

Parameter

-

Return value

```
{ "result": "<model-name>" }
```

e.g. : SP4K-25C

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/modelname
```

9.9 Get custom name projector

API

/rest/system/name

Type

GET

Parameter

-

Return value

```
{ "result": "<custom-name>" }
```

e.g. : PROJECTION-ROOM-1

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/name
```

9.10 Set custom name projector

API

/rest/system/name

Type

PUT

Parameter

vaue=<custom-name>

Return value

```
{}
```

Example (with curl)

```
curl --user username: pwd -X PUT http://<unit-ID>/rest/system/name? value=<custom-name>
```

9.11 Get serial number projector

API

/rest/system/serialnumber

Type

GET

Parameter

-

Return value

```
{ "result": "<serial-number>" }
```

e.g. : 2345678901

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/serialnumber
```

9.12 Get current state projector

API

/rest/system/state

Note: system.state can be accessed without passing user credentials.

Type

GET

Parameter

-

Return value

```
{ "result": "<current-state>" }
```

e.g. : error

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/state
```

9.13 Get the transition state of the device

API

/rest/system/targetstate

Type

GET

Parameter

-

Return value

```
{ "result": "<transition-state>" }
```

e.g. : ready

Example (with curl)

```
curl --user username:pwd http://<unit-ID>/rest/system/targetstate
```

9.14 Show activity

API

/rest/system/activity

Type

POST

Parameter

-

Return value

```
{ "result": "<activity>" }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/system/activity
```

9.15 Get identification of projector

API

/rest/system/getidentification

Type

POST

Parameter

params={"identification": “<identification-string>”}

E. g. : DisplayResolution, ArticleNumber, Identification, IdentificationFamily, Name, Productline, version, SerialNumber

Return value

```
{ "result": "<identification-result>" }
```

E.g. DCI4K

Example (with curl)

```
curl --user username: pwd -X POST http://<unit-ID>/rest/system/getidentification
--data-urlencode 'params={"identification":"<identification-string>" }'
```

9.16 Get identifications of projector

API

/rest/system/getidentifications

Type

POST

Parameter

-

Return value

Example

```
{
  "result": {
    "ArticleNumber": "R9008862",
    "DisplayResolution": "DCI4K",
    "Identification": "SP4K-25C",
    "IdentificationFamily": "SP4KC",
    "Name": "SP4K-25C-2345678901",
    "ProductLine": "SP4KC",
    "SerialNumber": "2345678901",
    "version": "1.6.0-dev-n1795"
  }
```

```
}
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/system/getidentifications
```

9.17 Go into eco mode

API

/rest/system/gotoeco

Type

POST

Parameter

-

Return value

```
{ }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/system/gotoeco
```

9.18 Go to ready mode

API

/rest/system/gotoready

Type

POST

Parameter

-

Return value

```
{ }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/system/gotoready
```

9.19 Power on light

API

/rest/system/poweron

Type

POST

Parameter

-

Return value

```
{ "result": { } }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/system/poweron
```

9.20 Power off light

API

/rest/system/poweroff

Type

POST

Parameter

-

Return value

```
{ "result": { } }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/system/poweroff
```

9.21 Reboot complete projector

API

/rest/system/reboot

Type

POST

Parameter

-

Return value

```
{ }
```

Example (with curl)

```
curl --user username:pwd -X POST http://<unit-ID>/rest/system/reboot
```

Index

A

Activity 59
Actual power
  Check 12
Article number
  Color wheel 55
  Projector 55
Availability
  Lens 45
Available
  Testpattern 19

C

Check
  Actual power 12
Clear
  Macro preset 29
CLO control 12
Color wheel
  Article number 55
  Name 55
Configuration method
  Device 38
Create
  Macro 28
Current lens 44
Current metadata 44
  Set 44
Current state
  Device 59
  Device in network 41
Custom
  Name 57–58

D

Device state
  Additional info 41
Domain name
  Network 37
Dowser
  Get object path 51
  Get position 50
  Get state 50–51
  Toggle 51
Dowser REST API 49

E

Eco mode
  Go to 61
Execute
  Macro 27
  Macro preset 27, 29

F

Family name 56
Fetch
  Executed macro 27
  Macro list 28
  Macro preset list 28
Firmware
  Version 56

G

Get name
  Testpattern 17
Get object path
  Dowser 51
Get state
  Dowser 51

H

Hostname
  Network 37

I

Identification
  Get 60
Identifications
  Get 60
Illumination REST API 11
Initial state
  Startup 56–57
Internal list
  Testpattern 21–22
Introduction 7–8
IP4 configuration
  Manual 39
  Network 39
IP6 configuration
  Manual 40
  Network 40
Iterate
  Testpattern 18
Iterate viewing order
  Testpattern 18–19

L

Lasers
  Power value
    Maximum 13
    Minimum 13
Lens
  availability 45
  Current lens 44
  Current metadata 44
    Set 44
  List
    Possible lenses 45
  Select 47
Lens REST API 43
Light
  Power off 62
  Power on 61
Limit reason
  Power value 14
Link
  Macro preset 30
List
  Lens 45
  Testpattern 20

M

MAC address
  Network 38
Macro
  Create 28
  Execute 27
  Fetch
    Executed macro 27
  Fetch preset list 28
  Preset
    Clear 29
    Clear all 29
    Execute 29
    Link 30
Macro list
  Fetch list 28
Macro preset
  Execute 27
Macro REST API 25
Mediaplayer
  Reset 33
  Serial number 32
  Status
    Availability 32
    Ready 32
  Type 33
Mediaplayer REST API 31
Model
  Name 57

N

Name
  Color wheel 55
  Custom 57–58
  Model 57
Network
  Configuration method
    Device 38
  Current state
    Device 41
  Device state
    Additional info 41
  Domain name 37
  General type
    Network device 38
  Hostname 37
  IP4 configuration 39
    Manual 39
  IP6 configuration 40
    Manual 40
  MAC address 38
  Service version 37
Network device
  General type 38
Network REST API 35

O

ON/OFF
  Testpattern 17

P

Position
  Dowser 50
Possible patterns
  Testpattern 23
Power
  Limited 12
Power off
  Light 62
Power on
  Light 61
Power value

Get 13
Lasers
Maximum 13
Minimum 13
Limit
Reason 14
Set 14
Projector
Article number 55
Propeties
Testpattern 20

R

Ready
Go to 61
Reboot
Device 62
Reset
Mediaplayer 33
REST API
Dowser 49
Lens 43
Macro 25
Mediaplayer 31
Network 35
Shutter 49
System 53
Testpattern 15
/rest/illumination/sources/laser/actualpower 12
/rest/illumination/sources/laser/controlledbyclo 12
/rest/illumination/sources/laser/ispowerlimited 12
/rest/illumination/sources/laser/maxpower 13
/rest/illumination/sources/laser/minpower 13
/rest/illumination/sources/laser/power 13–14
/rest/illumination/sources/laser/powerlimitreason 14
/rest/image/testpattern/available 19
/rest/image/testpattern/getproperties 20
/rest/image/testpattern/iterator/button/list 18–19
/rest/image/testpattern/iterator/button/listpossiblepatterns 23
/rest/image/testpattern/iterator/button/next 18
/rest/image/testpattern/iterator/button/size 22
/rest/image/testpattern/list 20
/rest/image/testpattern/listinternal 21
/rest/image/testpattern/selected 17–18
/rest/image/testpattern/setproperties 21
/rest/image/testpattern/show 17
/rest/image/testpattern/toggle 22
/rest/macro/create 28
/rest/macro/execute 27
/rest/macro/history 27
/rest/macro/list 28
/rest/macro/presets/clear 29
/rest/macro/presets/clearall 29
/rest/macro/presets/execute 27, 29
/rest/macro/presets/link 30
/rest/macro/presets/list 28
/rest/mediaplayer/available 32
/rest/mediaplayer/ready 32
/rest/mediaplayer/reset 33
/rest/mediaplayer/serialnumber 32
/rest/mediaplayer/type 33
/rest/network/device/lan/configuration 38
/rest/network/device/lan/devicetype 38
/rest/network/device/lan/hwaddress 38
/rest/network/device/lan/ip4config 39
/rest/network/device/lan/ip4configmanual 39
/rest/network/device/lan/ip6config 40
/rest/network/device/lan/ip6configmanual 40
/rest/network/device/lan/state 41
/rest/network/device/lan/stateinfo 41
/rest/network/domain 37
/rest/network/hostname 37
/rest/network/version 37
/rest/optics/lens/lens 44
/rest/optics/lens/list 45
/rest/optics/lens/metadata 44
/rest/optics/lens/select 47
/rest/optics/lens/selectavailable 45
/rest/optics/shutter/getobjectpath 51
/rest/optics/shutter/getstate 51
/rest/optics/shutter/position 50
/rest/optics/shutter/target 50
/rest/optics/shutter/toggle 51
/rest/system/activity 59
/rest/system/articlenumber 55
/rest/system/colorwheel 55
/rest/system/colorwheelname 55
/rest/system/familyname 56
/rest/system/firmwareversion 56
/rest/system/getidentification 60
/rest/system/getidentifications 60
/rest/system/gotoeco 61
/rest/system/gotoready 61
/rest/system/initialstate 56–57
/rest/system/modelname 57
/rest/system/name 57–58
/rest/system/poweroff 62
/rest/system/poweron 61
/rest/system/reboot 62
/rest/system/serialnumber 58
/rest/system/state 59
/rest/system/targetstate 59

S

Select
Lens 47
Testpattern 18
Serial number
Mediaplayer 32
Name 58
Service version
Network 37
Set
Power value 14
Shutter REST APi 49
Size list
Testpattern 22
Startup
Initial state 56–57
State
Current 59
Dowser 50

Transition 59
Status
Mediaplayer
Availability 32
Ready 32
Testpattern 17
Swagger UI 9
System REST API 53

T

Testpattern
Available 19
Get name 17
Internal list 21–22
Iterate 18
Iterate viewing order 18–19
List 20
List possible
Possible patterns 23
ON/OFF 17
Properties 20
Select 18
Size list 22
Status
ON/OFF 17
Testpattern REST API 15
Toggle
Dowser 51
Type
Mediaplayer 33

V

Version
Firmware 56